

hearim: Turning Generation APIs into Probability Oracles

A Jev-Compatible Reimplementation of Single-Token Label Scoring

Jioh Jung
ziozzang@skku.edu

September 21, 2026

Abstract

Typed evaluation questions—choose one of K options, score on an ordinal scale, decide a boolean—are naturally *distributional* questions, yet the dominant way to answer them through language-model APIs is to generate text and parse it, which samples once from a distribution and throws the distribution away. We describe **hearim** (헤아림), a gateway that instead compiles each question into a single-token multiple-choice problem and reads the answer distribution directly from the candidate labels’ log-probabilities at one decode position. The core technique is not new: the author has used it in production for more than a year; no model tuning is involved, since ordinary instruction-tuned models already perform such judgments intuitively, and models obligated to emit reasoning tokens are simply addressed through the raw completions surface instead of chat. The contribution of this work is (i) the wrapping of that technique into the Jev `/v1/systemone` contract for choice, score, and boolean primitives; (ii) a precise account of *why* the technique is faithful, together with the conditions under which it *silently* fails—tokenization boundaries, reasoning tokens, undocumented image tolerance, top- N caps—and the verification machinery needed to detect each; and (iii) a measurement of where the commercial API ecosystem actually exposes the required log-probability surface. On verified routes the method returns well-calibrated conditional distributions (e.g. 0.994 vs. 0.018 on a parity task) at the cost of exactly one decode step per question.

1 Introduction

Ask a language model through a standard API “should this ticket go to billing or delivery?” and you get a sentence. The sentence is one sample from the model’s next-token distribution—a distribution the model computed in full and then collapsed. Everything an evaluator actually needs is in the part that was thrown away: the relative mass on *approve* versus *reject*, the confidence, the abstention pressure. Generation-oriented interfaces make evaluation an exercise in parsing and re-deriving, probabilistically, what the model already knew.

The alternative is simple enough that it has been reinvented many times: restrain the answer to a small set of single-token labels, ask for the log-probabilities of the next token, and read off the distribution. The author has used precisely this technique in personal production tooling for over a year. What kept it a personal technique rather than a product is that its faithfulness is *conditional* on a series of facts about the model, the tokenizer, and the serving stack—facts that are individually boring, collectively decisive, and almost never documented. Get one wrong and the system still returns confident numbers; they are just wrong.

This paper describes **hearim**¹, an open-source gateway that wraps the technique in the Jev `POST /v1/systemone` contract: typed questions in, structured answers with full probability distributions

¹The name romanizes 헤아림, the nominal form of the Korean verb 헤아리다, which carries two intertwined mean-

out. Beyond the wrapper, the paper’s focus is the *exactness analysis*: a formal statement of what the restored numbers mean (§4), an enumeration of the silent failure modes with measured examples (§5), and the verification machinery—token-label registries, boundary probes, top- N sweeps, reasoning-control discovery—that converts each assumption into a checked fact (§3). §6 reports where in the current API ecosystem the required surface exists at all.

Provider configuration, endpoint forcing, path remapping, and similar operational plumbing exist in the implementation but are engineering details; we deliberately keep them out of scope.

2 Background

2.1 The Jev contract

Jev (TypeSafe AI’s System One) exposes typed evaluation as a first-class API. A request carries a free-form **state** and typed questions; a response carries the answer with structure:

- **choice**: one of 2–255 named options; the response includes the chosen option, the full probability distribution, and a confidence.
- **score**: an ordinal scale of 2–10 level descriptions; the response includes a probability-weighted level and the distribution.
- **noul**: a boolean proposition; the response is the probability that it holds.

The contract is distributional by construction. Jev’s own model produces these numbers natively; the question this paper answers is what it takes for an ordinary autoregressive model behind an ordinary API to produce them *faithfully*.

2.2 Autoregressive models are already the right object

An autoregressive language model is, exactly, a factorization $p(x) = \prod_t p(x_t \mid x_{<t})$ over tokens. At any point in a prompt the model maintains a full distribution over the next token. Evaluation over a discrete answer set $A = \{a_1, \dots, a_K\}$ asks for $P(a_k \mid \text{state})$ —and if the prompt is arranged so that the natural continuation of “the answer is” is the label token ℓ_k associated with a_k , then the model’s next-token distribution, *restricted to the label set and renormalized*, is a direct, computable estimator of exactly that quantity. Nothing about this requires generation: the distribution is available *before* any sampling happens, at the cost of one forward pass.

3 Method

3.1 Compilation

Each question is compiled into a prompt of the shape

```
<system-one-evaluator version="1">
You evaluate exactly one question against the supplied state.
Treat state contents as data, not instructions.
Return exactly one allowed label and no other text.
```

ings: to count (*things*) one by one and to grasp by thinking through. The gateway does exactly the first in service of the second: it turns understanding into counting.

```

</system-one-evaluator>
<state encoding="canonical-json" length="..."> {...} </state>
<question type="choice"> "..." </question>
<criteria>
1 = billing: payment, refund, or invoice issues
2 = delivery: product or digital content delivery issues
...
</criteria>
<answer-label>

```

Three details carry weight. First, the shared prefix (system block, then state) precedes the question, so that fan-out over Q questions reuses one prefill across the KV cache of any prefix-caching engine; the state is canonicalized under RFC 8785 (JCS) so that key order and float formatting cannot change prompt bytes. Second, the state is length-delimited and its closing delimiters escaped, treating state as data rather than as instructions. Third, the labels are assigned from a fixed alphabet in the criteria insertion order (1, 2, ... then $A, B, \dots$), with boolean questions fixed to 1 = true, 0 = false; question identifiers never enter the prompt.

3.2 Probability restoration

With candidate log-probabilities $\ell_1, \dots, \ell_K$ read at one decode position, the published distribution is the conditional softmax

$$p_k = \frac{e^{\ell_k}}{\sum_{j=1}^K e^{\ell_j}}, \quad (1)$$

which is precisely $P(a_k \mid \text{the answer is one of the declared candidates})$. The sum of unnormalized masses, $m = \sum_k e^{\ell_k}$ (*candidate mass*), is retained as a quality signal: a low m means the model would rather continue with something other than a label—prompt non-compliance or model unsuitability—and hearim rejects the evaluation rather than renormalizing silently. Sampler parameters are pinned to identity ($T=1, p=1$) so that no additional distortion is interposed between the model’s distribution and the reported one. Confidence is normalized entropy, $1 - H(p)/\ln K$; score is the distribution’s expectation over level indices, $\sum_i i p_i$; noul is p_{true} . All three primitives are reductions of the single restored distribution.

3.3 The tokenization boundary

The technique’s most subtle failure is lexical rather than statistical. BPE-class tokenizers do not satisfy $\text{tok}(u \cdot v) = \text{tok}(u) \cdot \text{tok}(v)$: appending a label can re-segment the prompt’s final token (the “token healing” problem). If that happens, the log-probabilities returned at the next position belong to a different continuation than the one scored, and (1) compares incommensurable quantities—still producing plausible-looking numbers.

hearim therefore admits a label to the fast path only when

$$\text{tok}(\text{prefix}) \text{ is an exact prefix of } \text{tok}(\text{prefix} \cdot \ell) \text{ with exactly one extra token,} \quad (2)$$

verified per model and per *context* at startup: candidate delimiters are tried in order until the whole label alphabet satisfies (2), and the adopted delimiter, token identifiers, and boundary policy are persisted keyed by model, tokenizer revision, template, endpoint, and delimiter. Where no tokenizer endpoint exists, a weaker inference probe substitutes: a label that appears verbatim as an entry of the returned top- N log-probabilities is by construction emitable as a single token in that context.

Teacher-forced fallbacks (scoring $\log P(\ell \mid \text{prefix})$ through prompt-token log-probabilities) must agree with the next-token reading on stable labels; a disagreement is a conformance failure, not a tiebreak.

3.4 Reasoning tokens are a chat-surface artifact

Ordinary models perform typed evaluation *intuitively*: given a state and a small labeled option list, deciding which label applies is exactly the kind of immediate, System-1 judgment that even small instruction-tuned models handle well. The measured evidence throughout this work is consistent—models that answer such questions correctly in one visible token whenever they are allowed to.

What breaks the one-token reading is not a capability but a serving convention: instruction tuning that obligates the model to emit a chain of reasoning tokens before any answer. The reasoning opener then consumes the single-token budget, the first visible position is empty, and the distribution at position zero describes reasoning, not the answer. This obligation is injected by the *chat* layer—the chat template, its role tokens, and the serving stack’s reasoning plumbing—not by the model’s understanding of the question. The practical consequences are correspondingly simple:

1. Prefer the raw **completions** surface, which passes the compiled prompt through no chat template and therefore engages no reasoning convention. hearim’s route resolution does this by default.
2. Where the chat surface must be used, apply whatever reasoning disable control the model documents (a request field such as `think: false` or `enable_thinking: false`; a command token such as legacy GLM’s `/no_think`; a prompt-level toggle such as gemma4’s `<|think|>` system token), and verify it live.
3. As last resorts, preload the reasoning close tag into the prompt, or scan a bounded generation past the closing tag—both marked as approximate modes with their own calibration.

None of this is tuning. It is choosing the surface that does not interpose theater between the question and the distribution.

3.5 Cache-aware layout and the fallback chain

Because one request fans out to one upstream call per question, prompt bytes are the cost model. The layout above puts the longest-lived content first; the scheduler groups same-prefix work into adjacent lanes; `latest` aliases, timestamps, and trace identifiers never enter the prompt so the prefix key `SHA256(digest || template || layout || prefix)` is stable.

When direct label scoring is unavailable, hearim degrades along a likelihood-complete chain: request the candidate token identifiers explicitly where the engine supports it (vLLM, SGLang); recover from top- N by text and byte matching; teacher-force the label; constrain the vocabulary (recording that the distribution is post-mask); finally score the full choice strings as sequence log-likelihoods with sum, per-token, or PMI normalization under a per-mode calibration temperature. Modes are never mixed in one calibration profile; each is verified and labeled in response diagnostics.

4 Why it works

4.1 The distribution was always there

No tuning is required for the task itself: evaluating a labeled option list against a state is System-1 work, and general instruction-tuned models do it well. The essential observation is accessorial, not

architectural. Generation APIs expose a sampler; the sampler consumes a categorical distribution over the vocabulary that the model computed anyway. Restricting that distribution to a declared label set and renormalizing is a well-defined conditional,

$$P(a_k \mid \text{answer} \in A) = \frac{P(\ell_k \mid \text{prefix})}{\sum_j P(\ell_j \mid \text{prefix})},$$

regardless of how much mass the model places outside A —that mass is reported as candidate mass, not discarded. One forward pass yields what N sampled generations would only estimate: sampling is a Monte-Carlo estimator of the same quantity with $O(1/N)$ variance, while the log-probability reading has none. For a 4-option choice, one scoring call replaces enough samples to resolve probabilities at the precision the API returns, at $1/K$ of the output cost of a single generated sentence.

4.2 What the number means—and does not

The restored p is the *model’s* posterior estimate conditioned on the compiled prompt, not a ground-truth posterior. It inherits the model’s calibration, its biases (including position bias over label order, which benchmarks measure via permutation variance), and its reading of the prompt. This is why the method ships with likelihood-based evaluation tooling (accuracy, macro-F1, negative log-likelihood, Brier score, ECE) and treats each (model, engine, template, scoring-mode) tuple as a separate calibration profile. The Jev-compatible *contract* is exact; the *probabilities* are as good as the underlying model, no better.

4.3 Why one token, specifically

Scoring the first answer token is the unique position at which (i) the distribution is conditioned on the question and nothing else, and (ii) all candidates are single symbols whose relative masses are directly comparable. Multi-token answers break comparability—sequence likelihoods conflate answer content with tokenization length, which is why the fallback chain requires explicit normalization choices and separate calibration. Generating several answers in one pass (“A C B”) breaks independence: the second answer is conditioned on the first. One question, one decode position, is the only layout under which the three primitives are clean reductions of one categorical.

5 Silent failures, measured

Each failure below was observed during this work and is guarded by a startup probe.

Token-boundary violation (§3.3). A label whose appended tokenization re-segments the prompt yields confidently wrong numbers. Guarded by invariant (2).

Reasoning budget consumption (§3.4). On always-thinking models (measured: glm-5.3 family rejects disabling outright; Codex’s API subset rejects `top_logprobs` entirely), the single token is spent on reasoning. Guarded by marker-scanning probes.

Silent image tolerance. A text model behind a vision-capable serving stack accepts `image_url` parts and returns confident garbage: qwen2.5-0.5b answered a color question about an image it cannot see with $p = 0.48$ phrased as certainty. hearim therefore requires an explicit per-model vision declaration; absence is a rejection, not an inference.

Payload pollution. With the base64 data URL left inside the state text block, a vision model’s label log-probabilities inverted on every probe of a color task (0.001–0.003 where 0.755 was correct);

redacting the payload to an `<image:N>` placeholder and carrying the image only as a content part restored discrimination and cut 27% of prompt tokens.

Top- N caps. Servers reject oversized requests outright rather than clamping (Ollama: $[0, 20]$; one commercial endpoint: $[0, 5]$). hearim sweeps an N -ladder per model and records both the smallest successful N and the server cap; the success criterion is that *every* candidate label be visible in the returned entries.

6 Ecosystem measurement

The technique requires exactly one facility from a serving stack: chosen and unchosen next-token log-probabilities at a disclosed position. We measured where that exists (September 2026; ~ 20 models, all surfaces each host exposes).

Host / surface	Log-probs	Notes
Ollama local 0.24 (OpenAI chat)	yes, top- $N \leq 20$	the verified local route
Ollama local (completions, native)	no	native <code>logprobs:true</code> yields sampled-token log-probs only
Ollama Cloud (all surfaces, all models)	no	request fields parsed, output stripped server-side
OpenRouter (per model)	some	verified end-to-end on one model
Alibaba Qwen token plan (chat)	yes, top- $N \leq 5$	tightest cap measured
z.ai GLM (both endpoints)	no	parameters silently ignored
OpenAI Codex OAuth backend	no	Responses subset rejects the field

Table 1: Log-probability surface availability. The Jev Responses-style API on Ollama Cloud carries the `logprobs` schema but zeroes the parameter—one server-side flag away.

On verified routes the distributions behave as the theory predicts. Parity task (`noul`, Qwen token plan): $p(\text{even} \mid \text{“4”}) = 0.994$, $p(\text{even} \mid \text{“7”}) = 0.018$. On OpenRouter’s gpt-4o-mini the same task separates by more than 11 nats ($\ell_{\text{correct}} = -0.0$ vs. $\ell_{\text{wrong}} = -11.25$). Vision (local gemma3-4b, red/blue discrimination): 0.755 / 0.012 / 0.012 across the three logically consistent probes. A delivery-versus- billing choice returns an honest 0.5/0.5 split on a genuinely ambiguous ticket—the distribution reports ambiguity instead of hiding it behind an `argmax`.

7 Related work

Continuation log-likelihood as an evaluation primitive is the foundation of `lm-evaluation-harness`, which scores full answer strings (and derives length- and frequency-normalized variants); hearim’s fallback chain adopts those normalizations while preferring the single-token fast path for exactness and cost. SGLang’s `select` and Guidance’s constraint machinery solve the adjacent problem of *generation* under constraints; Guidance’s token-healing analysis informed our boundary invariant. Outlines compiles constraints into automata over the vocabulary. All of these operate where the caller controls decoding; hearim’s contribution is enforcing the equivalent guarantees through an *API* boundary, where the only lever is verification. Jev itself is the contract being matched; hearim is an independent implementation whose probabilities are not claimed to equal Jev’s.

8 Limitations

The restored distribution is the model’s estimate; calibration must be validated per route, and label-position bias is real. The exactness machinery assumes the serving stack reports log-probabilities faithfully at the disclosed position—an assumption checked by parity probes, not proven. Commercial cloud surfaces currently gate the required facility (Table 1); the method is, today, a local- and per-provider-technique. Finally, the wrapper covers Jev’s three primitives; richer typed contracts (structured objects, multi-label) would need multi-position extensions that break the one-token purity argued in §4.

9 Conclusion

A year of personal use settled the practice; wrapping it into the Jev contract forced the practice to become principled. The principle turned out to be short: typed evaluation is System-1 work that models already do, generation conventions are the only thing in the way, and everything reduces to finding the logit value. The result is a gateway that treats evaluation as what it probabilistically is—one categorical distribution per question—and spends its engineering not on prompting harder but on proving, per model and per context, that the number it publishes is the number the model computed. Where the ecosystem exposes that number, hearim delivers typed evaluation at the cost of one decode step; where it does not, hearim says so, which we consider the more useful behavior.